Name: AKASH PRAKASH

Register No: 212224240008

Experiment 1

Implementation of Bayesian Networks

Aim : To create a bayesian Network for the given dataset in Python ## Algorithm: Step 1:Import necessary libraries: pandas, networkx, matplotlib.pyplot, Bbn, Edge, EdgeType, BbnNode, Variable, EvidenceBuilder, InferenceController
Step 2:Set pandas options to display more columns
Step 3:Read in weather data from a CSV file using pandas
Step 4:Remove records where the target variable RainTomorrow has missing values
Step 5:Fill in missing values in other columns with the column mean
Step 6:Create bands for variables that will be used in the model (Humidity9amCat, Humidity3pmCat, and WindGustSpeedCat)
Step 7:Define a function to calculate probability distributions, which go into the Bayesian Belief Network (BBN)
Step 8:Create BbnNode objects for Humidity9amCat, Humidity3pmCat, WindGustSpeedCat, and RainTomorrow, using the probs() function to calculate their probabilities
Step 9:Create a Bbn object and add the BbnNode objects to it, along with edges between the nodes
Step 10:Convert the BBN to a join tree using the InferenceController
Step 11:Set node positions for the graph
Step 12:Set options for the graph appearance
Step 13:Generate the graph using networkx
Step 14:Update margins and display the graph using matplotlib.pyplot

Program:

```
!pip install pybbn

import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
from pybbn.graph.dag import Bbn
from pybbn.graph.edge import Edge, EdgeType
from pybbn.graph.jointree import EvidenceBuilder
from pybbn.graph.node import BbnNode
from pybbn.graph.variable import Variable
from pybbn.pptc.inferencecontroller import InferenceController

pd.options.display.max_columns=50

df = pd.read_csv('/content/weatherAUS (1).csv', encoding='utf-8')
df = df[pd.isnull(df['RainTomorrow']) == False]
```

```
num_cols = df.select_dtypes(include='number').columns
cat_cols = df.select_dtypes(exclude='number').columns

df[num_cols] = df[num_cols].fillna(df[num_cols].mean())
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])

df['WindGustSpeedCat'] = df['WindGustSpeed'].apply(lambda x: '0.<=40' if x <= 40 else '1.40-50'
if 40 < x <= 50 else '2.>50')
df['Humidity9amCat'] = df['Humidity9am'].apply(lambda x: '1.>60' if x > 60 else '0.<=60')
df['Humidity3pmCat'] = df['Humidity3pm'].apply(lambda x: '1.>60' if x > 60 else '0.<=60')

print(df.head())

def probs(data, child, parent1=None, parent2=None):
    if parent1 is None:
        prob = pd.crosstab(data[child], 'Empty', margins=False,
normalize='columns').sort_index().to_numpy().reshape(-1).tolist()
    elif parent1 is not None:
        if parent2 is None:
            prob = pd.crosstab(data[parent1], data[child], margins=False,
normalize='index').sort_index().to_numpy().reshape(-1).tolist()
        else:
            prob = pd.crosstab([data[parent1], data[parent2]], data[child], margins=False,
normalize='index').sort_index().to_numpy().reshape(-1).tolist()
    else:
        print("Error in Probability Frequency Calculations")
    return prob

H9am = BbnNode(Variable(0, 'H9am', ['<=60', '>60']), probs(df, child='Humidity9amCat'))
H3pm = BbnNode(Variable(1, 'H3pm', ['<=60', '>60']), probs(df, child='Humidity3pmCat',
parent1='Humidity9amCat'))
W = BbnNode(Variable(2, 'W', ['<=40', '40-50', '>50']), probs(df, child='WindGustSpeedCat'))
RT = BbnNode(Variable(3, 'RT', ['No', 'Yes']), probs(df, child='RainTomorrow',
parent1='Humidity3pmCat', parent2='WindGustSpeedCat'))

bbn = Bbn() \
    .add_node(H9am) \
    .add_node(H3pm) \
    .add_node(W) \
    .add_node(RT) \
    .add_edge(Edge(H9am, H3pm, EdgeType.DIRECTED)) \
    .add_edge(Edge(H3pm, RT, EdgeType.DIRECTED)) \
    .add_edge(Edge(W, RT, EdgeType.DIRECTED))

join_tree = InferenceController.apply(bbn)

pos = {0: (-1, 2), 1: (-1, 0.5), 2: (1, 0.5), 3: (0, -1)}

options = {
    "font_size": 16,
    "node_size": 4000,
    "node_color": "pink",
    "edgecolors": "blue",
```

```
    "edge_color": "green",
    "linewidths": 5,
    "width": 5,
}

n, d = bbn.to_nx_graph()
nx.draw(n, with_labels=True, labels=d, pos=pos, **options)

ax = plt.gca()
ax.margins(0.10)
plt.axis("off")
plt.show()
```

Output:

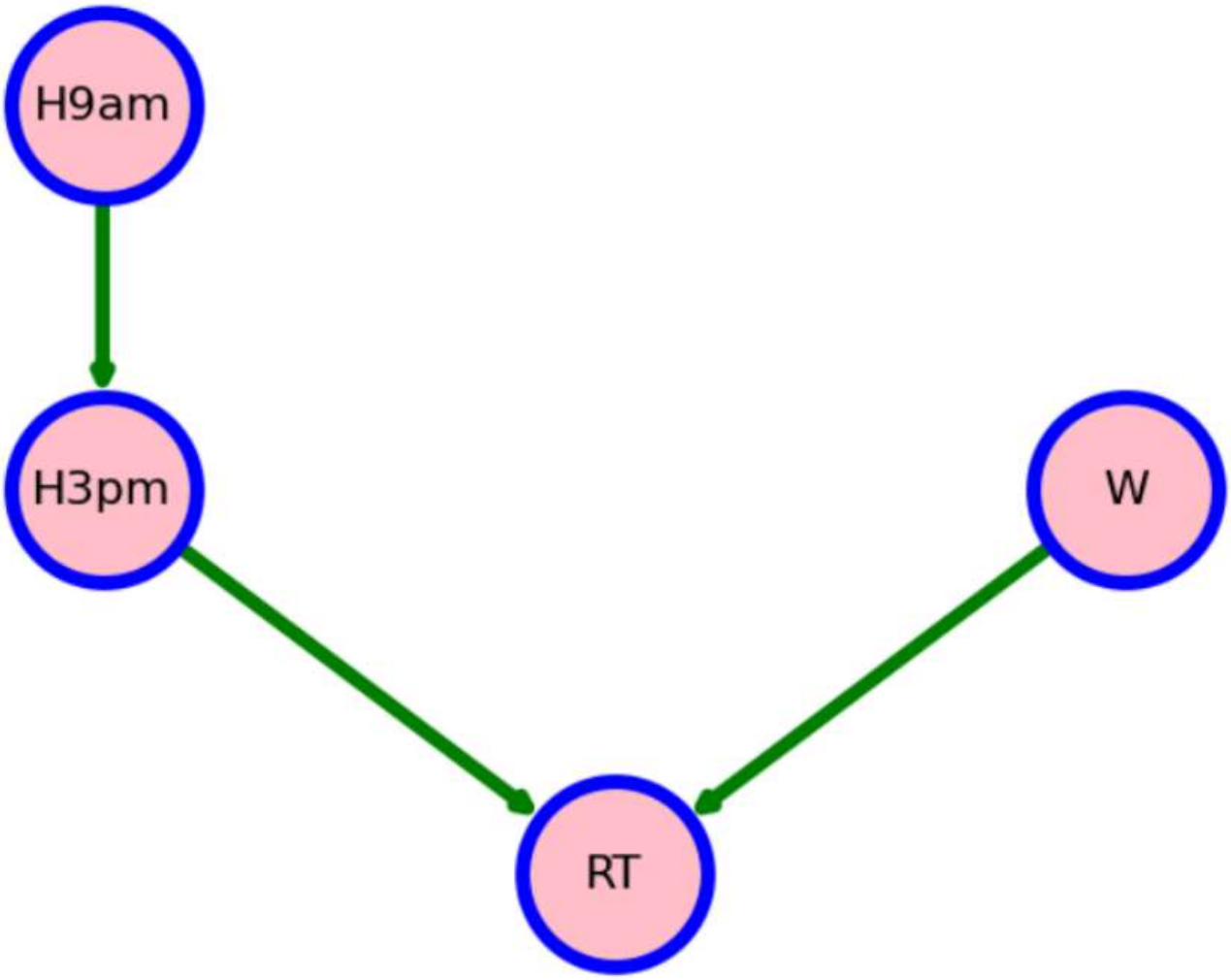
```
...
         Date Location  MinTemp  MaxTemp  Rainfall  Evaporation  Sunshine  \
0  2008-12-01   Albury     13.4     22.9       0.6     5.469824  7.624853
1  2008-12-02   Albury      7.4     25.1       0.0     5.469824  7.624853
2  2008-12-03   Albury     12.9     25.7       0.0     5.469824  7.624853
3  2008-12-04   Albury      9.2     28.0       0.0     5.469824  7.624853
4  2008-12-05   Albury     17.5     32.3       1.0     5.469824  7.624853

  WindGustDir  WindGustSpeed WindDir9am WindDir3pm  WindSpeed9am  \
0           W           44.0          W        WNW          20.0
1         WNW           44.0        NNW        WSW           4.0
2         WSW           46.0          W        WSW          19.0
3          NE           24.0         SE          E          11.0
4           W           41.0        ENE         NW           7.0

   WindSpeed3pm  Humidity9am  Humidity3pm  Pressure9am  Pressure3pm  Cloud9am  \
0          24.0         71.0         22.0       1007.7       1007.1  8.000000
1          22.0         44.0         25.0       1010.6       1007.8  4.437189
2          26.0         38.0         30.0       1007.6       1008.7  4.437189
3           9.0         45.0         16.0       1017.6       1012.8  4.437189
4          20.0         82.0         33.0       1010.8       1006.0  7.000000

   Cloud3pm  Temp9am  Temp3pm RainToday RainTomorrow WindGustSpeedCat  \
0  4.503167     16.9     21.8        No           No          1.40-50
1  4.503167     17.2     24.3        No           No          1.40-50
2  2.000000     21.0     23.2        No           No          1.40-50
3  4.503167     18.1     26.5        No           No           0.<=40
4  8.000000     17.8     29.7        No           No          1.40-50

  Humidity9amCat Humidity3pmCat
0          1.>60         0.<=60
1        0.<=60          0.<=60
2        0.<=60          0.<=60
3        0.<=60          0.<=60
4          1.>60         0.<=60
```



Result:

Thus a Bayesian Network is generated using Python